

目录

1	语言简介.....	1
2	工具及流程.....	2
2.1	工具安装.....	2
2.2	工程构建.....	2
3	基础组件.....	6
3.1	基础数据类型.....	6
3.2	基础信号类型.....	6
3.3	基础操作.....	6
3.4	基础表达.....	7
3.4.1	模块定义.....	7
3.4.2	选择逻辑.....	7
3.4.3	模块调用.....	8
4	常用语法表达.....	10
4.1	寄存器声明.....	10
4.1.1	单比特寄存器.....	10
4.1.2	32 个单比特寄存器.....	10
4.1.3	位宽为 32 位的寄存器.....	10
4.1.4	32 个位宽为 32 位的寄存器.....	10
4.1.5	Bundle 类型寄存器.....	10
4.1.6	32 个 Bundle 类型寄存器.....	11
4.1.7	带赋值逻辑的寄存器声明.....	11
4.2	信号的拼接与重复.....	11
4.3	模块实例化.....	11
4.3.1	实例化一个.....	12
4.3.2	实例化多个.....	12
5	工具库.....	17
5.1	Reverse.....	17

5.1.1	功能.....	17
5.1.2	调用示例.....	17
5.2	PopCount.....	17
5.2.1	功能.....	17
5.2.2	调用示例.....	17
5.3	OHToUInt.....	18
5.3.1	功能.....	18
5.3.2	调用示例.....	18
5.4	UIntToOH.....	18
5.4.1	功能.....	18
5.4.2	调用示例.....	18
5.5	PriorityEncoder	19
5.5.1	功能.....	19
5.5.2	调用示例.....	19
5.6	PriorityEncoderOH.....	19
5.6.1	功能.....	19
5.6.2	调用示例.....	19
5.7	Mux1H.....	19
5.7.1	功能.....	19
5.7.2	调用示例.....	20
5.8	MuxCase.....	20
5.8.1	功能.....	20
5.8.2	调用示例.....	20
5.9	PriorityMux	20
5.9.1	功能.....	20
5.9.2	调用示例.....	21
6	模块库.....	22
6.1	Queue.....	22
6.1.1	模块功能.....	22

6.1.2	可配参数.....	22
6.1.3	模块接口.....	22
6.1.4	调用示例.....	23
6.2	Arbiter	23
6.2.1	模块功能.....	23
6.2.2	可配参数.....	24
6.2.3	模块接口.....	24
6.2.4	调用示例.....	24
6.3	RRArbiter	25
6.3.1	模块功能.....	25
6.3.2	可配参数.....	25
6.3.3	模块接口.....	25
6.3.4	调用示例.....	25
7	自定义工具库.....	27
8	自定义模块库.....	28

1 语言简介

Chisel 是开源的硬件描述语言，它是基于 scala 语言构建出来的新的工具语言，它利用高级编程语言的灵活和高效去表达逻辑或者组件，借助编译器来转化为 verilog 硬件描述语言。如果 verilog 是用于生成电路的，那 Chisel 就是用于生成 verilog 的，它是 verilog 之上的一层抽象，最直观的体现就是更少的代码量。

2 工具及流程

2.1 工具安装

Chisel 是用 scala 语言构建出来的工具语言, scala 语言源于 java, 且 scala 以 java 虚拟机(JVM)为运行环境。所以使用 Chisel 语言必须先安装 Java OpenJDK(版本 11)。

Linux ubuntu 系统下安装 java OpenJDK 版本 11 的命令:

```
$ sudo apt install openjdk-11-jdk
```

检查是否已经安装 JDK 及其版本信息:

```
$ java -version
```

除此之外我们还需要 scala 的工程构建工具, 工具有很多, 这里我们使用 mill。

Linux ubuntu 系统下 mill 的安装命令:

```
$ sh -c "curl -L https://github.com/com-lihaoyi/mill/releases/download/0.9.8/0.9.8 > ~/.local/bin/mill && chmod +x ~/.local/bin/mill"
```

检查是否已经安装 Mill 及其版本信息:

```
$ mill --version
```

2.2 工程构建

chisel 工程本质是 scala 工程, mill 解析工程所需依赖文件是通过 build.sc 文件。我们以一个 demo 工程为例子。

创建一个名字叫 demo 的文件夹, 在文件夹下创建一个 build.sc 文件, 编写文件内容如图 2.1 所示。

- ✓ 2-8 行的 import 导入了需要的依赖包
- ✓ 10 行 object demo 定义了工程的名字 demo, extends SbtModule 决定了工程目录的组织结构如图 2.2 所示

- ✓ 11 行 millSourcePath 给出工程内容所在路径为 build.sc 所在目录
- ✓ 12 行 scalaVersion 给出所使用 scala 的版本为 2.12.13
- ✓ 22-24 行指出了工程需要依赖 Chisel3 的 3.5.0 版本的内容

```
1 // import Mill dependency
2 import mill._
3 import mill.define.Sources
4 import mill.modules.Util
5 import mill.scalalib.TestModule.ScalaTest
6 import scalalib._
7 // support BSP
8 import mill.bsp._
9
10 object demo extends SbtModule { m =>
11   override def millSourcePath = os.pwd
12   override def scalaVersion = "2.12.13"
13   override def scalacOptions = Seq(
14     "-language:reflectiveCalls",
15     "-deprecation",
16     "-feature",
17     "-Xcheckinit",
18   )
19   override def scalacPluginIvyDeps = Agg(
20     ivy"edu.berkeley.cs::chisel3-plugin:3.5.0",
21   )
22   override def ivyDeps = Agg(
23     ivy"edu.berkeley.cs::chisel3:3.5.0",
24   )
25 }
```

build.sc [+]

图 2.1 demo 工程的 build.sc 文件

```
demo
├── build.sc
└── src
    ├── main
    │   └── scala
```

图 2.2 sbtModule 定义的目录结构

在当前工程目录结构下，所有我们自己写的源文件都存放在 src/main/scala 目录下。因为 Chisel 只是 scala 构建出来的语言工具，我们实际还是在用 scala 语言编程，所以所有的源文件后缀名都是 .scala。

假设我们要在 demo 工程下写一个加法器， 那么就在 scala 目录下新建一个 Adder.scala 文件， 然后编写如图 2.3 所示内容。

```
1 import chisel3._
2
3 class Adder extends Module {
4   val io = IO(new Bundle{
5     val a = Input(UInt(8.W))
6     val b = Input(UInt(8.W))
7     val c = Output(UInt(8.W))
8   })
9   io.c := io.a + io.b
10 }
11
12 object Main extends App {
13   emitVerilog(
14     new Adder,
15     Array("--target-dir", "build")
16   )
17 }
18
~
src/main/scala/Adder.scala
```

图 2.3 Adder.scala

- ✓ 1 行， import chisel3._ 引入了 chisel3 包
- ✓ 3 行， 定义了一个 Adder 模块； 4-8 行； 模块的接口方向及位宽； 9 行， 加法器的内部逻辑
- ✓ 12 行， object Main extends App， 拓展了 App 的单列对象是默认的工程执行的入口
- ✓ 13 行， 调用 emitVerilog 函数生成对应模块的 verilog 代码； 模块名是 Adder； 指定了生成的 verilog 文件存放在 build 目录下

目前为止， 我们已经构建了一个简单的 Chisel 工程了。在 build.sc 所在路径调用 mill 去处理这个工程就能在 build 目录下看到生成的 verilog 文件了， 如图 2.4。

mill demo

```
1 module Adder
2   input      clock,
3   input      reset,
4   input [7:0] io_a,
5   input [7:0] io_b,
6   output [7:0] io_c
7 ;
8   assign io_c = io_a + io_b; // @[Adder.scala 9:16]
9 endmodule
~
build/Adder.v [+]
```

图 2.4 Chisel 代码生成的 verilog 文件

3 基础组件

3.1 基础数据类型

- ✓ Bool: 表示逻辑的真假，只有两个常数值，true.B 和 false.B
- ✓ UInt: 多比特的无符号数，需要指定位宽，可以用不同进制表达常数，比如 8bit 数 31 的表达：“b111_11”.U(8.W), “o37”.U(8.W), 31.U(8.W), “h1f”.U(8.W)
- ✓ Vec: 多个同类数据的集合 比如 Vec(8, Bool()), Vec(8, UInt(8.W)), 使用从 0 开始的编号对元素进行索引
- ✓ Bundle: 多个任意数据的集合，每个元素都有名字和数据类型的声明，使用元素名字进行索引

3.2 基础信号类型

chisel 只有四种信号类型，分别是 Wire/Reg/Input/Output。一个信号的表达包含信号类型和数据类型。

```
val a = Input(UInt(8.W))      //一个 8bit 的输入端口
val b = Wire(Bool())         //一个 1bit 的信号线
val c = Reg(UInt(8.W))       //一个 8bit 的寄存器
val d = Reg(Vec(8, UInt(8.W))) //一个 8x8bit 的寄存器组
val e = Output(UInt(8.W))    //一个 8bit 的输出信号
```

3.3 基础操作

Operator	Description	Data types
* / %	multiplication, division, modulus	UInt, SInt
+ -	addition, subtraction	UInt, SInt
== !=	equal, not equal	UInt, SInt, returns Bool
> >= < <=	comparison	UInt, SInt, returns Bool
<< >>	shift left, shift right (sign extend on SInt)	UInt, SInt
~	NOT	UInt, SInt, Bool

& ^	AND, OR, XOR	UInt, SInt, Bool
&&	logical AND, OR	Bool
v.andR v.orR v.xorR	AND, OR, XOR reduction	UInt, SInt, returns Bool
v(n)	extraction of a single bi	UInt, SInt
v(end, start)	bitfield extraction	UInt, SInt
Fill(n, v)	bitstring replication, n times	UInt, SInt
a ## b	bitfield concatenation	UInt, SInt
Cat(a, b, ...)	bitfield concatenation	UInt, SInt

表 3-1 Chisel 定义的信号操作

3.4 基础表达

3.4.1 模块定义

```
class Adder extends Module {
  val io = IO(new Bundle{
    val a = Input(UInt(8.W))
    val b = Input(UInt(8.W))
    val c = Output(UInt(8.W))
  })
  io.c := io.a + io.b
}
```

3.4.2 选择逻辑

Mux

Mux 对应于 Verilog 的三目运算

```
//Verilog: c = select? a : b
val select = Wire(Bool())
val a = Wire(UInt(8.W))
val b = Wire(UInt(8.W))
val c = Wire(UInt(8.W))
c := Mux(select, a, b)
```

When

When/.elseWhen/.otherwise: 对应于 Verilog 的 if/else if/else

```

when (cond) {
  w := 1.U
}.elsewhen (cond2) {
  w := 2.U
}.otherwise {
  w := 3.U
}

```

switch

switch 对应于 verilog 的 case 语句

```

switch(sel) {
  is (0.U) { result := 1.U}
  is (1.U) { result := 2.U}
  is (2.U) { result := 4.U}
  is (3.U) { result := 8.U}
}

```

3.4.3 模块调用

```

class CompA extends Module {
  val io = IO(new Bundle {
    val a = Input(UInt(8.W))
    val b = Input(UInt(8.W))
    val x = Output(UInt(8.W))
    val y = Output(UInt(8.W))
  })
  // function of A
}

class CompB extends Module {
  val io = IO(new Bundle {
    val in1 = Input(UInt(8.W))
    val in2 = Input(UInt(8.W))
    val out = Output(UInt(8.W))
  })
  // function of B
}

class CompC extends Module {
  val io = IO(new Bundle {
    val in_a = Input(UInt(8.W))
    val in_b = Input(UInt(8.W))

```

```
    val in_c = Input(UInt(8.W))
    val out_x = Output(UInt(8.W))
    val out_y = Output(UInt(8.W))
  })
  // create components A and B
  val compA = Module(new CompA())
  val compB = Module(new CompB())
  // connect A
  compA.io.a := io.in_a
  compA.io.b := io.in_b
  io.out_x := compA.io.x
  // connect B
  compB.io.in1 := compA.io.y
  compB.io.in2 := io.in_c
  io.out_y := compB.io.out
}
```

4 常用语法表达

4.1 寄存器声明

4.1.1 单比特寄存器

```
val xxxR = Reg(Bool()) //不带初始化  
val xxxR = RegInit(false.B) //带初始化
```

4.1.2 32 个单比特寄存器

```
val xxxR = Reg(Vec(32, Bool())) //不带初始化  
val xxxR = RegInit(VecInit.fill(32)(false.B)) //带初始化，且初始化的值  
相同  
val xxxR = RegInit(VecInit((0 until 32).map(i => if i > 16 false.B  
else true.B)))) //带初始化，初始化值跟具体元素相关
```

4.1.3 位宽为 32 位的寄存器

```
val xxxR = Reg(UInt(32.W)) //不带初始化  
val xxxR = RegInit(0.U(32.W)) //带初始化
```

4.1.4 32 个位宽为 32 位的寄存器

```
val xxxR = Reg(Vec(32, UInt(32.W))) //不带初始化  
val xxxR = RegInit(VecInit.fill(32)(0.U(32.W))) //带初始化，且初始化的值  
相同  
val xxxR = RegInit(VecInit((0 until 32).map(i => i.U(32.W)))) //带初  
始化，初始化的值跟具体元素有关
```

4.1.5 Bundle 类型寄存器

```
Class ABundle extends Bundle {  
    val valid = Bool()  
    val id = UInt(32.W)  
}  
val xxxR = Reg(new ABundle) //不带初始化  
val xxxR = RegInit(0.U.asTypeOf(new ABundle)) //带初始化
```

4.1.6 32 个 Bundle 类型寄存器

```
Class ABundle extends Bundle {  
  val valid = Bool()  
  val id = UInt(32.W)  
}  
val xxxR = Reg(Vec(32, new ABundle)) //不带初始化  
val xxxR = RegInit(VecInit.fill(32)(0.U.asTypeOf(new ABundle))) //带  
初始化
```

4.1.7 带赋值逻辑的寄存器声明

```
val xxxR1 = RegNext(xxxR) //不带初始化  
val xxxR1 = RegNext(xxxR, 0.U) //带初始化  
Note: RegNext 可以用于给任意数据类型的信号打拍子，包括 Vec 和 Bundle 类型
```

4.2 信号的拼接与重复

```
val xxx = Cat(Fill(2, "b110".U), "b00".U(2.W), aaa(3, 0), 4.U(4.W))
```

Note: "b00".U, 不管写几个零都是代表 1bit, 要表示多个 0 需要在后面加位宽信息

4.3 模块实例化

以下面的加法器模块为例

```
class Adder extends Module {  
  val io = IO(new Bundle{  
    val a = Input(UInt(8.W))  
    val b = Input(UInt(8.W))  
    val c = Output(UInt(8.W))  
  })  
  io.c := io.a + io.b  
}
```

4.3.1 实例化一个

在其他模块中实例化一个 Adder 模块，同时对端口信号进行连接。

```
val adder = Module(new Adder)
adder.io.a := a
adder.io.b := b
c := adder.io.c
```

4.3.2 实例化多个

在其他模块中实例化 4 个 Adder 模块，同时对端口信号都进行连接。

```
val adders = Seq.fill(4)(Module(new Adder))
for(i <- 0 until 4){
  adders(i).io.a := a(i)
  adders(i).io.b := b(i)
  c(i) := adders(i).io.c
}
```

或者

```
for(i <- 0 until 4) {
  val adder = Module(new Adder)
  adder.io.a := io.a(i)
  adder.io.b := io.b(i)
  io.c(i) := adder.io.c
}
```

4.4 Verilog 源码模块调用

Chisel 也可以直接嵌入一些现成的 Verilog 模块调用，Chisel 代码需要知道这些现成 verilog 模块的接口信息，具体 verilog 代码可以直接嵌入也可以通过路径指明代码位置，也可以完全不关系当作黑盒处理。

给出 verilog 代码内容或者路径信息的，在 build 目录下会出现 verilog 源

码， 否则只有 Chisel 生成的 verilog 代码。

4.4.1 直接嵌入 verilog 代码

```
import chisel3._
import chisel3.util._
import chisel3.experimental._

class FlashHelper extends HasBlackBoxInline {
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
    val data = Output(UInt(64.W))
    val addr = Input(UInt(32.W))
  })
  setInline("FlashHelper.v",
    s"""
      |import "DPI-C" function void flash_read
      |(
      |  input int addr,
      |  output longint data
      |);
      |
      |module FlashHelper (
      |  input clk,
      |  input [31:0] addr,
      |  input ren,
      |  output reg [63:0] data
      |);
      |
      |  always @(posedge clk) begin
      |    if (ren) flash_read(addr, data);
      |  end
      |
      |endmodule
      """.stripMargin)
  }

class AAA extends Module{
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
```



```

    val addr = Input(UInt(32.W))
    val data = Output(UInt(64.W))
  })
  val flash = Module(new FlashHelper)
  flash.io <> io
}

object Main extends App {
  emitVerilog(
    new AAA,
    Array("--target-dir", "build")
  )
}

```

4.4.2 指定 verilog 文件名

默认在 src/main/resource 路径下寻找指定的 Verilog 文件

```

import chisel3._
import chisel3.util._
import chisel3.experimental._

class FlashHelper extends HasBlackBoxResource {
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
    val data = Output(UInt(64.W))
    val addr = Input(UInt(32.W))
  })
  addResource("/FlashHelper.v")
}

class AAA extends Module{
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
    val addr = Input(UInt(32.W))
    val data = Output(UInt(64.W))
  })
  val flash = Module(new FlashHelper)
  flash.io <> io
}

```

```
object Main extends App {  
  emitVerilog(  
    new AAA,  
    Array("--target-dir", "build")  
  )  
}
```

4.4.3 指定 Verilog 文件路径

```
import chisel3._  
import chisel3.util._  
import chisel3.experimental._  
  
class FlashHelper extends HasBlackBoxPath {  
  val io = IO(new Bundle{  
    val clk = Input(Clock())  
    val ren = Input(Bool())  
    val data = Output(UInt(64.W))  
    val addr = Input(UInt(32.W))  
  })  
  addPath("/home/lyy/FlashHelper.v")  
}  
  
class AAA extends Module{  
  val io = IO(new Bundle{  
    val clk = Input(Clock())  
    val ren = Input(Bool())  
    val addr = Input(UInt(32.W))  
    val data = Output(UInt(64.W))  
  })  
  val flash = Module(new FlashHelper)  
  flash.io <> io  
}  
  
object Main extends App {  
  emitVerilog(  
    new AAA,  
    Array("--target-dir", "build")  
  )  
}
```

4.4.4 只关注 Verilog 模块接口

```
import chisel3._
import chisel3.util._
import chisel3.experimental._

class FlashHelper extends BlackBox {
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
    val data = Output(UInt(64.W))
    val addr = Input(UInt(32.W))
  })
}

class AAA extends Module{
  val io = IO(new Bundle{
    val clk = Input(Clock())
    val ren = Input(Bool())
    val addr = Input(UInt(32.W))
    val data = Output(UInt(64.W))
  })
  val flash = Module(new FlashHelper)
  flash.io <= io
}

object Main extends App {
  emitVerilog(
    new AAA,
    Array("--target-dir", "build")
  )
}
```

5 工具库

Chisel3 官方提供的特定逻辑功能， 不是独立模块。使用这些工具需要引入对应的 Chisel 包。

```
import chisel3.util._
```

5.1 Reverse

5.1.1 功能

逆序排列。输入比特序列， 输出逆序的比特序列。

5.1.2 调用示例

```
Reverse("b1101".U) // equivalent to "b1011".U  
Reverse("b1101".U(8.W)) // equivalent to "b10110000".U  
Reverse(myUIntWire) // dynamic reverse
```

5.2 PopCount

5.2.1 功能

对一个比特序列中 1 的个数进行统计。

5.2.2 调用示例

```
PopCount(Seq(true.B, false.B, true.B, true.B)) // evaluates to 3.U  
PopCount(Seq(false.B, false.B, true.B, false.B)) // evaluates to 1.U  
PopCount("b1011".U) // evaluates to 3.U  
PopCount("b0010".U) // evaluates to 1.U  
PopCount(myUIntWire) // dynamic count
```

5.3 OHToUInt

5.3.1 功能

通用编码器。

输入比特序列(Bits, UInt, Vec[Bool], Seq[Bool]), 返回 UInt 类型, 还可以指定返回数据的位宽。

5.3.2 调用示例

```
OHToUInt("b0100".U)    //返回 2.U
OHToUInt("b0100".U, 4)  //返回 2.U(4.W), 指定了返回数据位宽
val xxx = Wire(Vec(8, Bool()))
OHToUInt(xxx)           //传递的参数可以是 Vec[Bool]
OHToUInt((0 until 8).map(i => if i==0 true.B else false.B)) //传递的
参数是 Seq[Bool]
```

5.4 UIntToOH

5.4.1 功能

通用译码器。

输入 UInt 类型数据, 返回也是 UInt 类型数据, 还可以指定返回数据位宽。

5.4.2 调用示例

```
UIntToOH(2.U)  //不指定位宽
UIntToOH(2.U, 4) //指定位宽
```

5.5 PriorityEncoder

5.5.1 功能

优先级编码器， 返回首 1 的地址。

输入比特序列(Bits, UInt, Seq[Bool]), 输出 UInt 类型。

5.5.2 调用示例

```
PriorityEncoder("b0110".U) // results in 1.U
```

5.6 PriorityEncoderOH

5.6.1 功能

优先级编码电路， 返回只包含首 1 的独热码比特序列。

5.6.2 调用示例

```
PriorityEncoderOH(Seq(false.B, true.B, true.B, false.B)) // results in Seq(false.B,  
true.B, false.B, false.B)
```

5.7 Mux1H

5.7.1 功能

多路选择器。输入独热码比特序列和呆选数据序列， 输出独热码有效位对应的数据。

5.7.2 调用示例

输入键值对

```
val hotValue = chisel3.util.Mux1H(Seq(  
  io.selector(0) -> 2.U,  
  io.selector(1) -> 4.U,  
  io.selector(2) -> 8.U,  
  io.selector(4) -> 11.U,  
))
```

输入选择比特序列和待选择数据序列

```
ldQpc (i) := Mux1H(upVld, instIn.map(_.pc))
```

5.8 MuxCase

5.8.1 功能

优先级多路选择器。输入使能信号和数据对序列， 输出第一个是使能信号对应的数据。

5.8.2 调用示例

```
MuxCase(default, Array(c1 -> a, c2 -> b))
```

5.9 PriorityMux

5.9.1 功能

优先级多路选择器。 输入比特序列和待选数据序列， 输出比特序列第一个有效位对应的数据; 也可以输入是使能和数据序列对， 输出第一个是使能对应的数据。

5.9.2 调用示例

输入使能序列和数据序列

```
rdReqPReg := PriorityMux(evenRdReqVec, regChainOldReg.slice(LSB,  
MSB+1).reverse )
```

输入使能和数据序列对

```
pcChangeByJ.pc := PriorityMux(iRsp.map(i => i.valid && (i.isJ ||  
i.isBr) -> i.pc + i.imm))
```


6 模块库

Chisel3 官方提供的一些功能模块。使用这些模块需要引入对应的 Chisel 包。

```
import chisel3.util._
```

6.1 Queue

6.1.1 模块功能

可以存放任意 chisel 数据类型的 FIFO 队列。

6.1.2 可配参数

参数	类型	默认值	含义
gen	T	无	队列保存的数据类型
entries	Int	无	队列项数
pipe	Boolean	false	
flow	Boolean	false	
useSyncReadMem	Boolean	false	
hasFlush	Boolean	false	是否有 flush 端口

表 6-1 Queue 调用的参数列表

6.1.3 模块接口

信号名	方向	类型	含义
enq.ready	O	Bool	队列有空可接收数据
enq.valid	I	Bool	有数据要入队
enq.bits	I	任意	要入队的数据
deq.ready	I	Bool	出队数据可被接收
deq.valid	O	Bool	有数据可以出队
deq.bits	O	任意	要出队的数据
flush	I	Bool	可选端口， 清空队列

表 6-2 Queue 接口信号说明

6.1.4 调用示例

存储的数据类型为 UInt

```
val q = Module(new Queue(UInt(), 16))
q.io.enq <> producer.io.out
consumer.io.in <> q.io.deq
```

存储的数据类型为 Bundle, 带 flush 功能

```
class QBundle extends Bundle with CoreParameters{
  val isFlashAddr = Bool()
  val id          = UInt(IDWidth.W)
  val addr        = UInt(VAddrWidth.W)
  val data        = UInt(256.W)
  val bp          = Vec(8, UInt(2.W))
}
val rspQ = Module(new Queue(new QBundle, 2, hasFlush = true))
rspQ.io.enq.valid := i_r.valid && idReqVld(i_r.bits.id(IDWidth-1, 0))
rspQ.io.enq.bits := reqAddrQ.io.deq.bits
rspQ.io.enq.bits.data := i_r.bits.data
rspQ.io.enq.bits.bp := bp
rspQ.io.flush.get := pcChange.valid
rspQ.io.deq.ready := !stall
decode.valid := rspQ.io.deq.valid
decode.pc := rspQ.io.deq.bits.addr
decode.inst := rspQ.io.deq.bits.data
decode.bp := rspQ.io.deq.bp
```

6.2 Arbiter

6.2.1 模块功能

对多个请求进行仲裁, 选择一个请求出来, 选择算法是编号越小优先级越高。

6.2.2 可配参数

参数	类型	默认值	含义
gen	T	无	被仲裁数据类型
n	Int	无	被仲裁请求数目

表 6-3 Arbiter 可配参数列表

6.2.3 模块接口

信号名	方向	类型	含义
in(i).ready	O	Bool	表示第 i 个请求被选中
in(i).valid	I	Bool	表示第 i 个请求有效
in(i).bits	I	任意	表示第 i 个请求的数据
out.ready	I	Bool	表示请求可以被接收
out.valid	O	Bool	表示有请求输出
out.bits	O	任意	表示输出请求的数据
chosen	O	UInt	表示被选中请求的编号

表 6-4 Arbiter 接口信号说明

6.2.4 调用示例

被仲裁数据是 UInt 类型

```
val arb = Module(new Arbiter(UInt(), 2))
arb.io.in(0) <> producer0.io.out
arb.io.in(1) <> producer1.io.out
consumer.io.in <> arb.io.out
```

被仲裁数据是 Bundle 类型

```
val arbRdAxi = Module(new Arbiter((new AxiArBundle), 2)) //dRd.ar
have higher priority
arbRdAxi.io.in(0) <> io.d_ar
arbRdAxi.io.in(1) <> io.i_ar
io.ar <> arbRdAxi.io.out
```

6.3 RRArbiter

6.3.1 模块功能

对多个请求进行仲裁， 选择一个请求出来， 选择算法是 round robin。

6.3.2 可配参数

参数	类型	默认值	含义
gen	T		被仲裁数据类型
n	Int		被仲裁请求数目

表 6-5 RRArbiter 可配参数列表

6.3.3 模块接口

信号名	方向	类型	含义
in(i).ready	O	Bool	表示第 i 个请求被选中
in(i).valid	I	Bool	表示第 i 个请求有效
in(i).bits	I	任意	表示第 i 个请求的数据
out.ready	I	Bool	表示请求可以被接收
out.valid	O	Bool	表示有请求输出
out.bits	O	任意	表示输出请求的数据
chosen	O	UInt	表示被选中请求的编号

表 6-6 RRArbiter 接口信号说明

6.3.4 调用示例

被仲裁数据是 UInt 类型

```
val arb = Module(new RRArbiter(UInt(), 2))
arb.io.in(0) <> producer0.io.out
arb.io.in(1) <> producer1.io.out
consumer.io.in <> arb.io.out
```

被仲裁数据是 Bundle 类型

```
val arbRdAxi = Module(new RRArbiter(new QBundle, LdNum))
for(i <- 0 until LdNum) {
  arbRdAxi.io.in(i) <> dRdQ(i).io.deq
}
```

```
arbRdAxi.io.out.ready := d_ar.ready  
d_ar.valid := arbRdAxi.io.out.valid  
d_ar.bits.id := arbRdAxi.io.chosen  
d_ar.bits.addr := arbRdAxi.io.out.bits.addr
```

7 自定义工具库

自定义特定逻辑功能，不是独立模块。使用这些工具需要引入对应的工具包。

```
import tools._
```

8 自定义模块库

9 编码风格

9.1 常量参数定义

- 常量参数首字母大写
- 表示数目的参数加 Num 后缀；表示位宽的参数加 Width 后缀

```
val IidNum = 128  
val IidWidth = log2Up(IidNum)
```

尽量不使用 ZipWithIndex, 编码不易于理解

10 错误记录